



UNSW
SYDNEY

COMP4141

Theory of Computation

Lecture 3: Properties of Regular Languages

Outline

Closure properties

Regular Operations

Closure under complement

Lemma

Let $L \subseteq \Sigma^$ be a regular language. Show that $\Sigma^* \setminus L$ is also a regular language.*

More closure properties

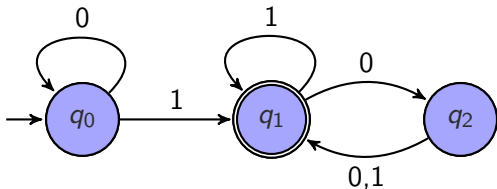
Theorem

Let $L_1, L_2 \subseteq \Sigma^$ be regular languages. The following languages are regular:*

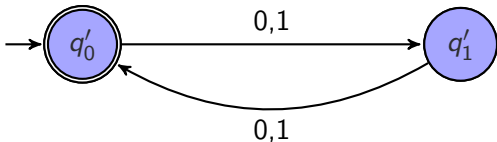
- $L_1 \cap L_2$
- $L_1 \cup L_2$
- $L_1 \setminus L_2$
- $L_1 \oplus L_2$ (symmetric difference)

Closure under intersection: Proof idea

Idea: Take DFAs that accept L_1 and L_2 ; move through them *simultaneously*



w : 1001 ✓



Closure under intersection: Product construction

Let $A_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ be a DFA that accepts L_1 and $A_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ be a DFA that accepts L_2 . Define $A = (Q, \Sigma, \delta, q_0, F)$ as follows:

- $Q = Q_1 \times Q_2$
- $\delta((q, q'), a) = (\delta_1(q, a), \delta_2(q', a))$
- $q_0 = (q_1, q_2)$
- $F = F_1 \times F_2 = \{(q, q') : q \in F_1 \text{ and } q' \in F_2\}$

Claim

$$L(A) = L(A_1) \cap L(A_2)$$

Closure under intersection: Product construction

Suppose $w \in L(A)$. Let $(s_0, s'_0)(s_1, s'_1) \dots (s_k, s'_k)$ be the accepting run of w in A . So $s_0 = q_1$ and $s'_0 = q_2$; and $s_k \in F_1$ and $s'_k \in F_2$. From the definition of δ we have that $s_0 s_1 \dots s_k$ is an accepting run of w in A_1 and $s'_0 s'_1 \dots s'_k$ is an accepting run of w in A_2 . So $w \in L(A_1) \cap L(A_2)$.

Suppose $w \in L(A_1) \cap L(A_2)$. Let $s_0 s_1 \dots s_k$ be the accepting run of w in A_1 , so $s_0 = q_1$ and $s_k \in F_1$. Let $s'_0 s'_1 \dots s'_k$ be the accepting run of w in A_2 , so $s'_0 = q_2$ and $s'_k \in F_2$. Therefore $(s_0, s'_0) = q_0$, and $(s_k, s'_k) \in F$. From the definition of δ we have that $(s_0, s'_0)(s_1, s'_1) \dots (s_k, s'_k)$ is an accepting run of w in A ; so $w \in L(A)$.

Closure under union, set difference, symmetric difference

Once we have complement and intersection, the rest is straightforward:

- $L_1 \cup L_2 = \Sigma^* \setminus ((\Sigma^* \setminus L_1) \cap (\Sigma^* \setminus L_2))$
- $L_1 \setminus L_2 = L_1 \cap (\Sigma^* \setminus L_2)$
- $L_1 \oplus L_2 = (L_1 \setminus L_2) \cup (L_2 \setminus L_1)$

Outline

Closure properties

Regular Operations

Regular Expressions (next lecture)

Regular expressions are an *algebraic notation* for regular languages.

Extensively used

- String pattern matching utilities (e.g. `grep` in unix)
- Computer language definitions (“lexical structure”)
- Compiler generation tools

Regular expressions are a completely different “formalism” from finite automata — but they have *exactly the same expressive power*.

Tools based on regular expressions usually translate them to finite automata, which are more suitable than regular expressions for many applications.

Concatenation of Languages

If L_1 and L_2 are languages, we can define their *concatenation* L_1L_2 to be $\{ xy \mid x \in L_1 \wedge y \in L_2 \}$.

Examples

What is $\{ab, ba\}\{cd, dc\}$?

What is $\emptyset\{ab, bc\}$?

Exponentiation of Languages

L^i is the language L concatenated with itself i times.

Definition

Base: $L^0 = \{\epsilon\}$.

Induction: $L^{i+1} = LL^i$.

Examples

$$\{ab, ba\}^2 = \{abab, abba, baab, baba\}$$

$$\emptyset^0 = \{\epsilon\}$$

$$\emptyset^2 = \emptyset$$

Kleene Closure

$$L^* = \bigcup_{i \in \mathbb{N}} L^i = L^0 \cup L^1 \cup \dots$$

This operator can build infinite sets from finite sets.

Examples

$$\{ab, ba\}^* = \{\epsilon, ab, ba, abab, abba, \dots\}$$

$$\emptyset^* = \{\epsilon\}$$

$$\{\epsilon\}^* = \{\epsilon\}$$

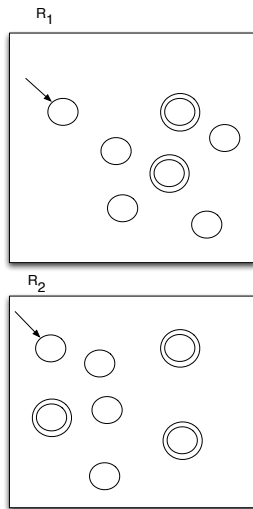
Proof exercise

Theorem

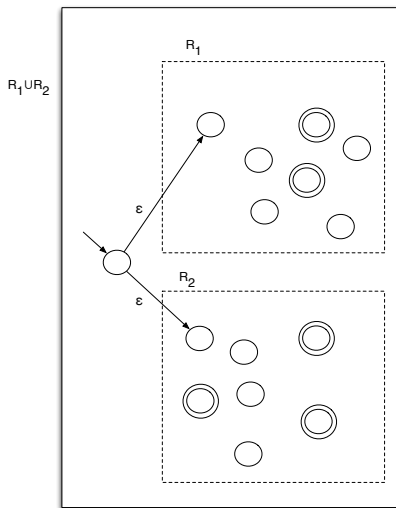
Regular languages are closed under:

- *Union*
- *Concatenation*
- *Kleene closure*

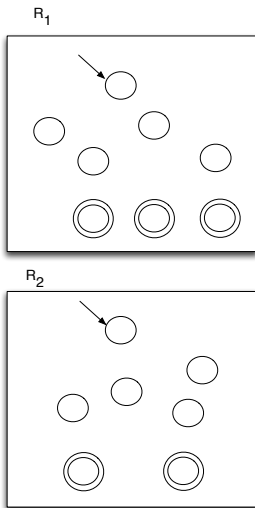
Proof: Closure under union



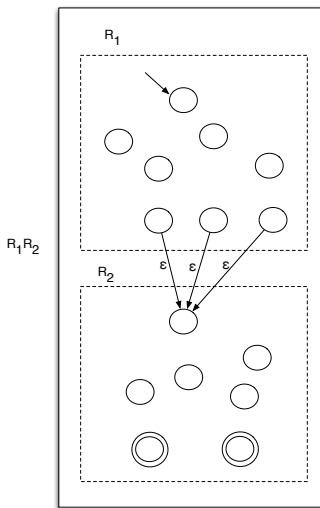
Proof: Closure under union



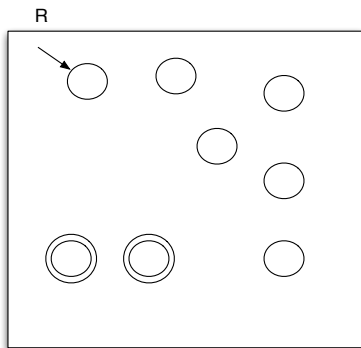
Proof: Closure under concatenation)



Proof: Closure under concatenation



Proof: Closure under Kleene star



Proof: Closure under Kleene star

